

Kubernetes: o que é esse negócio que todo mundo fala

Emerson Silva

DOUGBR Meetup · [UNIFAAT](#) · Atibaia



Emerson Silva

- Engenheiro DevOps/SRE na **4Linux**
- +9 anos em ambientes DevOps críticos
- Foco em **Kubernetes**, IaC e confiabilidade
- Escritor, instrutor e palestrante ativo na comunidade





Fundada em 2001, a 4Linux participou das principais transformações da área de tecnologia. Começamos com Linux e software livre, navegamos por Cloud, DevOps e Containers — e hoje atuamos com agentes de IA, sempre com base em **inovação aberta**.



Sorteio 4Linux

Aponte a câmera e participe!



sorteio.4linux.com.br/event/6559d41b



O que veremos hoje

1. A evolução: bare metal → VMs → containers
2. O problema de gerenciar containers em escala
3. Como surgiu o Kubernetes
4. O que é e como funciona
5. Quando usar (e quando não usar)
6. Vantagens e certificações
7. Os principais objetos
8. Deploy na prática: Snake Classic no K8s

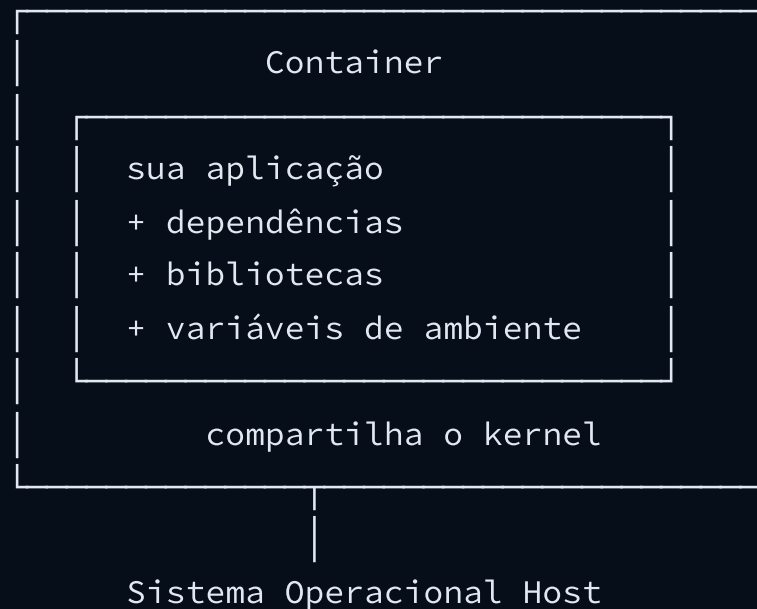
A evolução da infraestrutura

	Bare Metal	Máquinas Virtuais	Containers
<i>Isolamento</i>	Total (hardware)	Parcial (hypervisor)	Processo (kernel)
<i>Inicialização</i>	Minutos	Segundos a minutos	Milissegundos
<i>Overhead</i>	Zero	Médio (SO completo)	Mínimo
<i>Portabilidade</i>	Baixa	Média	Alta

“ Containers são rápidos, leves e portáveis – mas gerenciar **muitos** deles é um desafio

”

O que é um container?

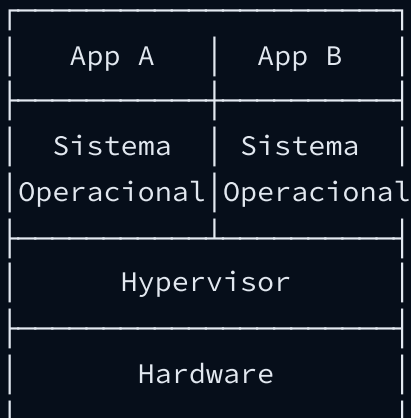


“ Um container empacota a aplicação **e tudo que ela precisa para rodar** — e roda de forma isolada no mesmo kernel do host

”

Container vs Máquina Virtual

Máquina Virtual



S0 completo por app (GBs)

Container



Só o necessário (MBs)

O problema de escala

Imagina sua aplicação rodando em **10 containers**. Tudo certo.

Agora imagina **200 containers**, em **8 servidores diferentes**:

- Em qual servidor sobe cada container?
- Como garantir que o container reinicia se cair?
- Como distribuir carga entre as réplicas?
- Como fazer update sem derrubar o serviço?
- Como os containers se comunicam entre servidores?

“ Fazer isso na mão é inviável. Você precisa de um **orquestrador**. ”

É aqui que entra o Kubernetes

Como surgiu o Kubernetes

- **2003** — Google cria o **Borg**, sistema interno de orquestração
- Borg gerenciava *bilhões* de containers por semana nos data centers do Google
- **2013** — Google inicia projeto baseado nas lições do Borg
- **Junho de 2014** — Kubernetes anunciado publicamente
- **2016** — Doador para a **CNCF** (Cloud Native Computing Foundation)
- **Hoje** — padrão da indústria, com contribuições de centenas de empresas

“ O nome vem do grego: **κυβερνήτης** — timoneiro, piloto de navio ”

O que é Kubernetes

“ **Kubernetes** é um sistema open source de orquestração de containers que automatiza o deploy, escalabilidade e gerenciamento de aplicações containerizadas.

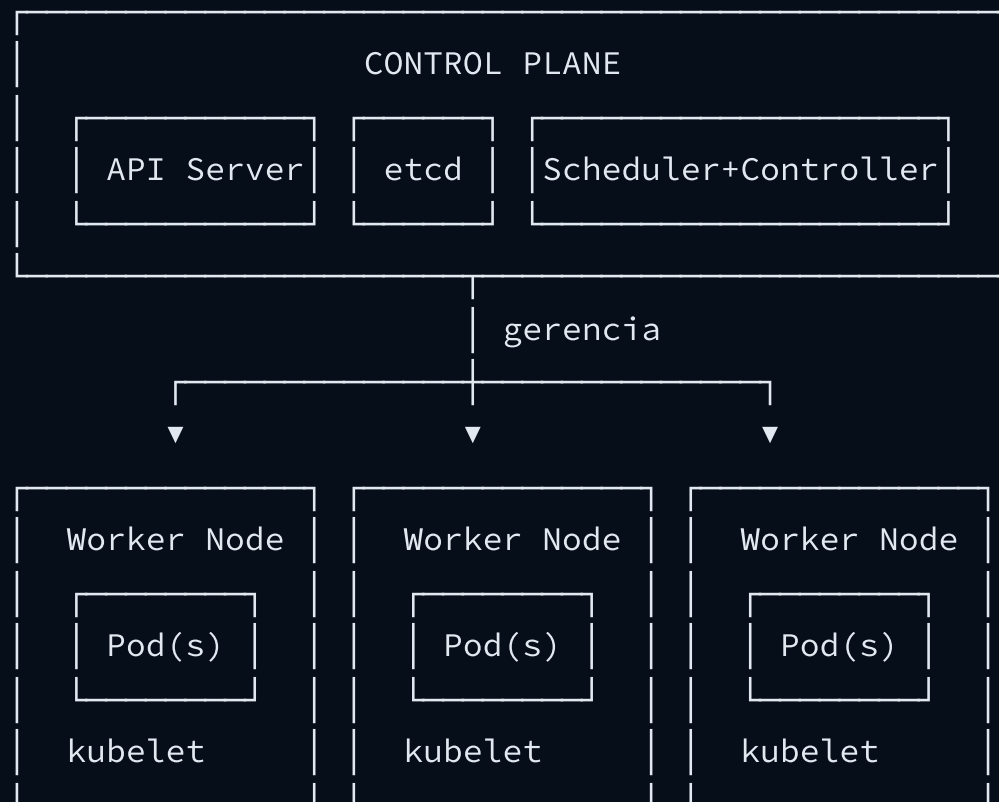
”

Em termos simples:

- Você diz **o que quer** — "quero 3 réplicas da minha app rodando"
- Kubernetes cuida do **como** — em qual nó, como manter, como recuperar

Kubernetes não substitui o Docker — ele usa o Docker (ou outro runtime) por baixo dos panos

Arquitetura: visão geral



Control Plane: o cérebro do cluster

- **API Server** — porta de entrada para tudo; toda interação passa por aqui (`kubectl` fala com ele)
- **etcd** — banco de dados distribuído; armazena o estado completo do cluster
- **Scheduler** — decide em qual nó cada Pod vai rodar
- **Controller Manager** — garante que o estado real bate com o desejado

“ Se você pede "quero 3 réplicas", o Controller Manager é quem percebe que caiu uma e sobe outra ”

Worker Node: onde as aplicações rodam

- **kubelet** — agente em cada nó; recebe ordens do control plane e garante que os containers estão rodando
- **kube-proxy** — cuida das regras de rede e roteamento de tráfego entre pods
- **Container Runtime** — quem realmente executa os containers (containerd, CRI-O)

“ O control plane **gerencia**, os worker nodes **executam**

”

Quando usar Kubernetes

Use Kubernetes quando...	Talvez não precise quando...
Muitos microsserviços para gerenciar	É um projeto pequeno ou MVP
Precisa de alta disponibilidade	Time pequeno, pouca experiência
Escala horizontal frequente	Monólito simples e estável
Times diferentes deployando em paralelo	Custo operacional é restrição crítica
Multi-cloud ou portabilidade é requisito	Uma VM simples resolve o problema

“ Kubernetes resolve problemas de escala. Se você não tem esse problema, talvez seja overkill. ”

Vantagens do Kubernetes

- **Self-healing** — se um container cai, o K8s reinicia automaticamente
- **Auto-scaling** — escala pods horizontalmente conforme a demanda
- **Rolling updates** — atualiza a aplicação sem downtime
- **Rollback** — voltou pra versão anterior em um comando
- **Portabilidade** — roda igual na AWS, GCP, Azure ou on-premise
- **Service discovery** — pods se encontram pelo nome, sem IP fixo
- **Gerenciamento de configuração** — ConfigMaps e Secrets nativos

| Os principais objetos do Kubernetes

Pod: a menor unidade

“ Um **Pod** é o menor objeto do Kubernetes — agrupa um ou mais containers que compartilham rede e armazenamento. ”

```
apiVersion: v1
kind: Pod
metadata:
  name: meu-app
spec:
  containers:
  - name: app
    image: nginx:1.25
    ports:
    - containerPort: 80
```

- Na prática, você quase nunca cria Pods diretamente
- Pods são criados e gerenciados por objetos de nível superior

Deployment: gerencia seus Pods

“ Um **Deployment** declara quantas réplicas de um Pod devem existir e como atualizar a aplicação. ”

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: meu-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: meu-app  # Liga o Deployment aos Pods pelo label
```

Deployment: definindo o container

```
template:          # molde de cada Pod que será criado
  metadata:
    labels:
      app: meu-app
  spec:
    containers:
      - name: app
        image: nginx:1.25
        ports:
          - containerPort: 80
```

- Se um Pod cair, o Deployment recria automaticamente
- Para escalar: `kubectl scale deployment meu-app --replicas=5`
- Para atualizar a imagem: `kubectl set image deployment/meu-app app=nginx:1.26`

Service: expõe seus Pods

“ Um **Service** cria um ponto de acesso estável para os Pods — os Pods mudam, o Service permanece. ”

```
apiVersion: v1
kind: Service
metadata:
  name: meu-app
spec:
  selector:
    app: meu-app
  ports:
    - port: 80
      targetPort: 80
  type: ClusterIP # interno | NodePort | LoadBalancer
```

Service: tipos de exposição

- **ClusterIP** — acesso só de dentro do cluster. Padrão. Ideal para comunicação entre serviços internos.
- **NodePort** — expõe uma porta fixa (30000–32767) em cada nó do cluster. Útil para testes e desenvolvimento.
- **LoadBalancer** — provisiona automaticamente um load balancer externo na nuvem (AWS ELB, GCP LB...). Usado em produção.

“ Para ambientes locais (minikube), use **NodePort** ou `kubectl port-forward` ”

Namespace: isolamento lógico

“ **Namespaces** dividem o cluster em ambientes isolados – útil para separar times, projetos ou ambientes. ”

```
kubectl get namespaces
# default      Active   5d
# kube-system   Active   5d    ← componentes do K8s
# kube-public   Active   5d
# monitoring    Active   2d    ← seu namespace
```

```
kubectl create namespace staging
kubectl apply -f app.yaml -n staging
```

“ Sem namespace especificado, tudo vai para o **default** ”

ConfigMap: configurações não-sensíveis

“ Separa a configuração do código – sem precisar rebuildar a imagem para mudar um valor. ”

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_ENV: "producao"
  LOG_LEVEL: "info"
  DB_HOST: "postgres-service"
```

ConfigMap: injetando no container

opção 1 – injeta todas as chaves de uma vez

envFrom:

- configMapRef:
 - name: `app-config`

opção 2 – injeta chave por chave

env:

- name: `APP_ENV`
 - valueFrom:
 - configMapKeyRef:
 - name: `app-config`
 - key: `APP_ENV`

“ Mudou a config? Atualiza o ConfigMap e reinicia os pods – sem rebuild de imagem. ”

Secret: dados sensíveis

“ Igual ao ConfigMap, mas para senhas, tokens e chaves. Valores em **base64**. ”

```
apiVersion: v1
kind: Secret
metadata:
  name: app-secret
data:
  DB_PASSWORD: c2VuaGEtc2VjcmV0YQ== # base64 de "senha-secreta"
  API_TOKEN: bXl0b2t1bnNlY3JldA==
```

```
# como gerar o valor base64
echo -n "senha-secreta" | base64
```

Secret: injetando e boas práticas

```
envFrom:  
- secretRef:  
  name: app-secret
```

- Base64 é **encoding**, não criptografia — qualquer um pode decodificar
- Nunca commite Secrets no Git em texto puro
- Para produção: use **Sealed Secrets** (criptografia no Git) ou **Vault** (gerenciador externo)
- Clouds oferecem alternativas nativas: AWS Secrets Manager, GCP Secret Manager

| Hora do deploy!

Vamos rodar o Snake Classic no Kubernetes

Deploy: silvemerson/snake-classic

```
# snake.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: snake-classic
spec:
  replicas: 1
  selector:
    matchLabels:
      app: snake-classic
  template:
    metadata:
      labels:
        app: snake-classic
    spec:
      containers:
        - name: snake-classic
          image: silvemerson/snake-classic:0.1
```

Expondo o jogo com um Service

```
apiVersion: v1
kind: Service
metadata:
  name: snake-classic
spec:
  selector:
    app: snake-classic
  type: NodePort
  ports:
    - port: 80
      targetPort: 8080
      nodePort: 30080
```

Comandos para subir o deploy

sobe o cluster kind com port mapping

```
kind create cluster --config kind/cluster.yaml
```

aplica Deployment e Service

```
kubectl apply -f kind/snake.yaml
```

acompanha o pod subindo

```
kubectl get pods -w
```

acessa via port-forward (alternativa)

```
kubectl port-forward service/snake-classic 8080:80
```


O que aconteceu por baixo dos panos

1. `kubectl apply` → API Server recebe o YAML
2. API Server → salva o estado desejado no etcd
3. Scheduler → escolhe um Worker Node disponível
4. kubelet → recebe a tarefa e instrui o containerd
5. containerd → faz pull da imagem `silvemerson/snake-classic`
6. Container → sobe e escuta na porta 8080
7. kube-proxy → configura o roteamento do NodePort
8. Service → balanceia o tráfego até o Pod

“ Tudo isso em segundos, de forma declarativa

”

Certificações Kubernetes (CNCF)

Certificação	Foco	Tipo
KCNA	Fundamentos de K8s e Cloud Native	Teórica
KCSA	Fundamentos de segurança Cloud Native	Teórica
CKAD	Desenvolver e fazer deploy de apps no K8s	Prática
CKA	Administrar e operar clusters K8s	Prática
CKS	Segurança avançada em Kubernetes	Prática

- Provas práticas: 2h resolvendo problemas em cluster real — sem múltipla escolha
- Reconhecidas globalmente · Recomendação: comece pela **KCNA** ou **CKAD**

KubeAstronaut

Quem conquista as **5 certificações** (KCNA + KCSA + CKAD + CKA + CKS) recebe o título de **KubeAstronaut** da CNCF — reconhecimento máximo do ecossistema Kubernetes

- Título vitalício renovável — reconhecido mundialmente pela comunidade CNCF
- Badge oficial que pode ser exibido no LinkedIn e currículo
- Acesso a uma comunidade exclusiva de especialistas Kubernetes
- Representa domínio completo: do desenvolvimento à operação e segurança

“ Uma jornada, não uma prova — cada cert tem seu valor independente ”

Conclusão

Surgiu no Google —
baseado no Borg,
open source em 2014

Orquestra containers
— cuida de deploy,
escala e recuperação

Declarativo — você
descreve o que quer,
ele resolve o como

Kubernetes é **complexo**, mas os primeiros passos são **simples** — e valem
muito a pena

EMERSON SILVA

Kubernetes para Iniciantes: Fundamentos e Práticas



Kubernetes para Iniciantes: Fundamentos e Práticas

Emerson Silva

Gratuito na Amazon - Mas amanhã, pq esqueci
de ativar pra hj :(

Escaneie para acessar na Amazon:



4.Linux

Open
Software
Specialists



Obrigado!

Por onde continuar

- 📖 kubernetes.io/docs — documentação oficial
- 🎮 killercode.com — labs interativos gratuitos
- 🎓 training.linuxfoundation.org — preparação CKA/CKAD
- 📝 emerson-silva.blog.br



Vamos manter contato!